NutriPet AI — Estrutura técnica e plano de construção

Documento de trabalho de 11 de outubro de 2026. Recomendação de arquitetura, stack e método para construir o NutriPet AI, partindo do Documento Conceitual v1.0 e da análise de mercado. São recomendações para decisão, não uma especificação final.

1. Princípios que guiam as escolhas

1. **Fundação antes de IA.** Banco, login, permissões e auditoria vêm primeiro; a IA entra depois, sobre dados confiáveis (mesma recomendação do documento conceitual).
2. **Histórico nunca é sobrescrito.** Registros clínicos são *append-only* com versões; correção gera nova versão, não apaga a anterior.
3. **Segurança no banco, não só na tela.** A regra "cada usuário vê só o que pode" deve valer mesmo se o front-end falhar ou alguém acessar uma URL direta.
4. **IA como rascunho.** Todo conteúdo da IA nasce com status "não aprovado" e só vira parecer após revisão humana explícita.
5. **Simples de operar por uma pessoa.** Menos serviços, uma linguagem principal, infraestrutura gerenciada sempre que possível.
6. **Trocar fornecedor sem reescrever.** IA, OCR e armazenamento ficam atrás de interfaces próprias.

2. Visão geral da arquitetura

Arquitetura de **monólito modular** (uma aplicação, módulos bem separados) com um **worker assíncrono** para tarefas pesadas. Microsserviços seriam complexidade desnecessária nesta fase.

Camada	Responsabilidade	Escolha recomendada
Front-end	Interface responsiva (celular e computador), formulários, ficha, dashboards, área do tutor	Aplicação web React/Next.js em modo PWA
Aplicação / API	Regras de negócio, validação, controle de acesso, orquestração dos módulos	TypeScript no mesmo projeto (rotas de servidor)

Camada	Responsabilidade	Escolha recomendada
Autenticação e permissões	Login, recuperação de acesso, papéis, sessões, vínculo profissional–paciente e tutor–animal	Provedor de autenticação estabelecido (não construir do zero) + RBAC no app + RLS no banco
Banco de dados	Dados relacionais, versões, auditoria	PostgreSQL gerenciado
Arquivos	Exames, relatórios, planos em PDF	Armazenamento de objetos privado com URLs temporárias assinadas
Fila e worker	OCR, extração por IA, geração de rascunhos, envio de lembretes	Fila de jobs + processo worker separado
IA e OCR	Extração estruturada, resumo, comparação	Camada de abstração sobre um ou mais provedores de modelos
Observabilidade	Logs, erros, auditoria, alertas	Logs estruturados + rastreamento de erros + trilha de auditoria em tabela

Por que TypeScript de ponta a ponta

Uma linguagem só para front e back reduz troca de contexto e facilita o desenvolvimento assistido por IA, com tipos compartilhados entre formulários, API e banco. Se no futuro o processamento de exames pedir bibliotecas de Python, ele pode viver como um worker à parte, sem afetar o resto.

Hospedagem

Preferência por região no Brasil (São Paulo), para latência e para facilitar a conversa de LGPD sobre onde ficam os dados. Duas rotas razoáveis:

- **Rota A, plataforma gerenciada (mais simples):** hospedagem de app + Postgres gerenciado + armazenamento S3-compatível. Menos operação, ideal para o MVP.
- **Rota B, nuvem de grande porte:** mais controle de rede, criptografia, backups e conformidade, com mais configuração. Faz sentido ao se aproximar de clientes pagantes e de auditoria.

Recomendação: começar na Rota A com tudo isolado atrás de interfaces, e migrar para a B quando houver volume e exigências de clientes. **Decidir região e fornecedor só depois de checar custos e termos atuais**, porque variam.

3. Modelo de dados (base do documento conceitual)

O modelo da seção 10 do documento conceitual é bom. Ajustes recomendados: **versão em todo registro clínico**, **chaves de vínculo explícitas** para permissão e **hash dos arquivos**.

Entidades principais

- `organizacao` (clínica ou profissional autônomo): ppode ter vários profissionais.
- `usuario` e `papel` (tutor, profissional, admin da clínica, admin da plataforma).
- `tutor`, `animal`, `vinculo_tutor_animal`, `vinculo_profissional_animal`.
- `atendimento` (tipo: consulta, consultoria, retorno, reavaliação) com `status` (rascunho, em revisão, finalizado).
- `medicao` (peso, condição corporal, medidas) com **unidade e método sempre gravados**.
- `plano_alimentar` e `plano_alimentar_item`, versionados.
- `documento_exame`, `observacao_exame`, `analise_ia`, `revisao_profissional`.
- `auditoria` (quem, o quê, quando, em qual entidade).

Padrão de versionamento

Tabela principal guarda o registro "atual"; tabela de versões guarda cada alteração com autor, data e motivo. Correções nunca fazem `UPDATE` silencioso no conteúdo clínico.

```
-- Esboço ilustrativo (não é o esquema final)
create table atendimento (
  id uuid primary key,
  animal_id uuid not null references animal(id),
  profissional_id uuid not null references usuario(id),
  tipo text not null check (tipo in
('consulta','consultoria','retorno','reavaliacao')),
  status text not null check (status in
('rascunho','em_revisao','finalizado')),
  ocorrido_em timestamptz not null,
  atendimento_origem_id uuid references atendimento(id),
  versao_atual int not null default 1,
  criado_em timestamptz not null default now()
);

create table atendimento_versao (
  atendimento_id uuid references atendimento(id),
  versao int,
  conteudo jsonb not null,          -- campos do formulário daquela versão
  autor_id uuid not null,
```

```
  motivo text,
  criado_em timestamptz not null default now(),
  primary key (atendimento_id, versao)
);

create table medicao (
  id uuid primary key,
  animal_id uuid not null references animal(id),
  atendimento_id uuid references atendimento(id),
  tipo text not null,               -- peso, escore_corporal,
circunferencia...
  valor numeric not null,
  unidade text not null,
  metodo text,
  medido_em timestamptz not null
);
```

Permissões no banco (Row Level Security)

Políticas de RLS no PostgreSQL garantem, por exemplo: tutor só lê animais vinculados a ele e apenas conteúdos *publicados*; profissional só lê pacientes com vínculo ativo. Isso cobre o critério de aceite "usuário sem permissão não acessa ficha nem arquivo por URL direta".

4. Pipeline de exames com IA

O módulo de maior risco. Fluxo recomendado:

1. **Upload** para armazenamento privado; gravar hash, tipo, datas, laboratório, vínculo com animal e atendimento.
2. **Fila:** o upload cria um job; o usuário não espera o processamento.
3. **Leitura do documento:** OCR para PDFs escaneados ou imagens; extração direta de texto quando o PDF já tiver texto.
4. **Extração estruturada** por modelo, com **esquema JSON rígido**: parâmetro, resultado original, unidade, referência, página/região de origem e campo de incerteza.
5. **Validação determinística (sem IA):** unidade ausente, número ilegível, duplicidade, referência incompatível com a espécie. O que falhar fica marcado como *incerto*, nunca preenchido por suposição.
6. **Análise preliminar** (opcional e posterior): resumo com evidências e limitações, sempre rotulada como rascunho.
7. **Tela de revisão:** valor extraído ao lado do trecho do documento original; profissional aprova, corrige ou rejeita campo a campo.

8. **Publicação:** relatório versionado que separa dado do documento, observação automatizada e parecer profissional.

Regras de segurança da IA

- Não tratar intervalo de referência como diagnóstico; não recomendar dieta terapêutica com base em exame isolado.
- Sinais de possível urgência vão para o profissional, sem decisão automática.
- Cada resultado guarda **versão do modelo, entrada usada e saída**, para rastreabilidade.
- Correções do profissional são armazenadas como *feedback*, mas **não alteram o modelo automaticamente**; entram numa fila de avaliação antes de qualquer mudança de prompt ou regra.
- Dados de tutores e animais não são usados para treinamento externo sem base legal, transparência e controles. Verificar, ao contratar o provedor de IA, as condições de retenção e uso de dados.

Como avaliar a qualidade

Montar um **conjunto de teste** com exames reais (autorizados e anonimizados) e o gabarito conferido por profissional. Medir, por campo: taxa de acerto, taxa de campos marcados como incertos e taxa de correção pelo revisor. Só liberar mudanças de modelo ou prompt se não piorarem essas métricas. Isso corresponde aos indicadores de "tempo de revisão e taxa de correção" do documento conceitual.

5. Segurança, LGPD e operação

- **Criptografia** em trânsito e em repouso; segredos fora do código.
- **Arquivos privados** sem links públicos permanentes; URLs assinadas com validade curta, geradas só após checagem de permissão.
- **Acesso mínimo** para admin da plataforma, excepcional e com registro de auditoria.
- **Auditoria** de leitura e escrita em dados sensíveis, em tabela *append-only*.
- **Direitos do titular:** exportar, corrigir e excluir/anonimizar dados do tutor conforme a lei, equilibrando com obrigações de retenção do prontuário (ponto para validar com advogado).
- **Consentimentos** registrados por finalidade e por canal (lembretes por mensagem, por exemplo).
- **Backups automatizados e teste de restauração** antes de produção; plano de resposta a incidentes escrito.
- **Testes de segurança:** revisão de permissões (tentar ler dados de outro tutor/paciente), teste de acesso por URL direta, análise de dependências.

- Revisão jurídica de LGPD, termos e limite de atuação profissional antes de qualquer lançamento comercial.

6. Fases de construção

As fases seguem o roadmap do documento conceitual, com entregas verificáveis. **As estimativas são aproximadas**, para uma pessoa com desenvolvimento assistido por IA e disponibilidade parcial; revisar após a primeira fase.

Fase	Entrega	Critério de pronto	Esforço aproximado
0. Validação e protótipo	Entrevistas com 10–15 profissionais; protótipo clicável da ficha, novo atendimento e evolução	Profissionais testam e dizem o que pagariam	2 a 4 semanas
1. Fundação	Projeto, banco, autenticação, papéis, RLS, auditoria básica, backup e CI	Dados persistem; cada usuário vê só o permitido; restauração testada	3 a 5 semanas
2. Prontuário (MVP vendável)	Cadastro de tutor/animal, os 4 tipos de atendimento, linha do tempo, medições, plano alimentar versionado, área do tutor	Histórico preservado e consultável por período	5 a 8 semanas
3. Exames	Upload privado, metadados, extração, tela de revisão e relatório versionado	Cada valor com origem e revisão registradas	5 a 8 semanas
4. IA controlada	Resumo de evolução, comparação temporal, rascunhos revisáveis, conjunto de avaliação	Testes e aprovação clínica documentados	4 a 6 semanas
5. Agenda e comunicação	Retornos, lembretes, importação/exportação com sistemas de gestão	Consentimentos e falhas tratados	3 a 5 semanas
6. Comercialização	Cobrança, onboarding, suporte, termos e políticas	Avaliação de segurança, jurídica e operacional concluída	4 a 6 semanas

Ponto de decisão importante: ao final da Fase 2 já existe um produto que um profissional consegue usar e pagar, sem exames nem IA. Se as entrevistas apontarem que exames são a maior dor, a Fase 3 pode subir de prioridade, mas sem pular a fundação.

7. Como construir na prática

7.1 Método de trabalho (desenvolvimento assistido por IA com governança)

- Manter um **arquivo de instruções do projeto (CLAUDE.md)** com: regras de negócio, padrões de código, regras de segurança, o que a IA *não* pode fazer (por exemplo, nunca fazer `UPDATE` em conteúdo clínico sem criar versão) e a lista de critérios de aceite.
- Você define a lógica e os critérios de qualidade; a IA acelera a implementação em passos pequenos, e cada passo termina com **teste automatizado** e revisão sua.
- Revisão humana obrigatória em três áreas: **permissões/RLS, migrações de banco e qualquer código que toque dados clínicos ou arquivos.**

7.2 Estrutura do repositório (sugestão)

```
nutripet/
  CLAUDE.md               # regras do projeto e critérios de aceite
  apps/
    web/                  # Next.js (profissional + tutor, PWA)
    worker/               # jobs: OCR, extração, rascunhos, lembretes
  packages/
    db/                   # esquema, migrações, políticas RLS, seeds
    domain/               # regras de negócio e validações compartilhadas
    ai/                   # interfaces e provedores de IA/OCR, prompts
versionados
    ui/                   # componentes comuns
  docs/                   # decisões de arquitetura (ADRs), modelo de dados
  tests/                  # testes de permissão, e2e, conjunto de avaliação
da IA
```

7.3 Primeiros passos concretos

1. Criar o repositório com o `CLAUDE.md` e registrar as decisões de arquitetura (um arquivo curto por decisão).
2. Subir banco, autenticação e ambiente de desenvolvimento e produção separados.
3. Escrever **primeiro os testes de permissão** (tutor A não vê animal do tutor B; profissional sem vínculo não vê paciente) e depois as políticas que os fazem passar.

4. Implementar cadastro de animal–tutor e o primeiro tipo de atendimento ponta a ponta (formulário, versionamento, linha do tempo).
5. Adicionar os demais tipos de atendimento reaproveitando o mesmo mecanismo.
6. Colocar o piloto na mão de 2 ou 3 profissionais reais o quanto antes e ajustar com o uso.

7.4 Testes e qualidade

- **Testes de permissão** automatizados em toda mudança (os mais importantes do projeto).
- Testes de unidade para regras de unidades, datas e comparações de medições.
- Testes ponta a ponta dos fluxos principais (criar animal, registrar retorno, comparar evolução).
- Conjunto de avaliação separado para extração de exames (Fase 3 em diante).
- Integração contínua rodando testes e verificação de dependências antes de cada publicação.

8. Integração com sistemas de gestão (SimplesVet e similares)

- **Não presumir** que exista API aberta ou que permita acesso a todos os dados; confirmar API oficial, escopos, contrato, limites e homologação.
- Começar com **importação e exportação controladas** (por exemplo, CSV de cadastro de animais e tutores; PDF/JSON dos relatórios aprovados).
- Definir, por tipo de informação, **qual sistema é a fonte principal** e como resolver duplicidades e conflitos.
- Isolar a integração em módulo próprio para poder trocar ou adicionar sistemas sem tocar no núcleo.

9. Custos e modelo de cobrança (o que levantar)

Não estimei valores, porque preços mudam e dependem de volume. Itens a cotar antes de fechar preço:

- Hospedagem do app e do worker; banco gerenciado; armazenamento e transferência de arquivos.
- Provedor de autenticação (se pago).
- OCR e modelos de IA (custo por documento processado: **medir no protótipo**).
- E-mail, mensagens e notificações.
- Monitoramento, backup e segurança.
- Jurídico (LGPD, termos) e eventual auditoria de segurança.

Hipótese de cobrança (a validar nas entrevistas): assinatura mensal por profissional ou clínica, com limites de pacientes e armazenamento; área do tutor incluída.

10. Decisões em aberto

Decisão	Opções	Quando decidir
Fornecedor de hospedagem e região	Plataforma gerenciada x nuvem de grande porte	Início da Fase 1
Provedor de autenticação	Serviço gerenciado x solução própria sobre padrão aberto	Início da Fase 1
Provedor de IA/OCR	Mais de um, atrás de interface comum; testar com exames anonimizados	Antes da Fase 3
Escopo clínico e regulatório	Limites entre apoio nutricional e ato veterinário	Antes do piloto com pacientes reais
Política de retenção e exclusão	Prazos de guarda do prontuário x direito de exclusão	Antes da produção
Integrações prioritárias	Qual sistema de gestão os primeiros clientes usam	Após as entrevistas
Aplicativo nativo ou PWA	PWA no início; nativo só se houver necessidade comprovada	Após a Fase 2

11. Resumo da recomendação

Monólito modular em TypeScript (Next.js em PWA), PostgreSQL com RLS e versionamento de registros, arquivos privados com URLs assinadas, worker com fila para OCR e IA, e IA sempre como rascunho com revisão humana. Construir em fases: validação, fundação, prontuário (primeiro produto vendável), exames e só depois IA ampliada, agenda e comercialização. Começar pelos testes de permissão e pelo primeiro tipo de atendimento ponta a ponta.

Limitação: as escolhas de tecnologia refletem práticas comuns e a prioridade de manter o projeto simples para uma pessoa. Não testei fornecedores nem cotei custos; vale confirmar versões, preços e termos atuais antes de contratar qualquer serviço.